

# ADG classes: Minhashes (MinHash, FracMinHash, ModHash)

Aleksander Jankowski

05.11.2024

Today we will again use data from metagenomic samples collected during the scientific expedition Tara Oceans (2009-2012). Seawater from different depths was filtered to retain small cell sizes (bacteria). The three datasets will come from different seas around the world. The DNA was extracted and submitted to high throughput sequencing. The files provided on Moodle contain a selection of reads that contain no missing (N) nucleotides. Each read is stored in separate line.

For hashing, we will use MurmurHash3 (`mmh3` Python package), you can install it in a new virtualenv as follows:

```
python3 -m venv .venv
source .venv/bin/activate
pip3 install mmh3
```

Hint: you may wish to work on a smaller subset of reads for most of the exercises below.

## 1 Min-hash.

1. Inspect the code that uses mini-hash and multiple hash functions to estimate Jaccard similarity coefficient.
2. Adapt the function `jaccard_min_kmer_hashes()` to estimate the Jaccard similarity between two selected samples, using  $k$ -mers of length 15. How does the estimate behave as you increase the number of saltings?
3. Consider several odd values of  $k$  in the procedure above. How does the choice of  $k$  influence the convergence speed?

## 2 MinHash

1. Write a program to estimate the Jaccard index between two selected samples, using  $k$ -mers of length 15 and  $s = 1000$  smallest hashes as a sketch.
2. How does the estimate of Jaccard index changes as you vary the value of  $s$  (say from 100 to 10,000)? Plot the resulting estimates.
3. Hint: you may use the `string_to_kmer_set()` function from last week:

```
def string_to_kmer_set(Astr, k):  
    return set([Astr[i:i+k] for i in range(len(Astr)-k+1)])  
  
string_to_kmer_set("hello", 3)
```

## 3 FracMinHash

1. Modify the code to estimate the Jaccard index using FracMinHash approach. Instead of keeping the  $s$  smallest hashes, keep as sketch all the hashes  $h$  such that  $h \leq H/s$ , where  $H$  is the maximal value of the hash function.
2. Try  $s = 100,000$  first, and observe how the estimate changes as you vary the value of  $s$ .
3. Hint: `mmh3.hash("hello", signed=False)` returns a 32-bit unsigned int.

## 4 ModHash

1. ModHash is similar to FracMinHash, but instead of keeping all the hashes  $h$  such that  $h \leq H/s$ , we retain the ones satisfying  $h \bmod s = 0$ .
2. Are there any advantages and disadvantages of ModHash compared to FracMinHash? Hint: they are not that different!